

Async JavaScript and fetch

Module 2 - Lesson 3

Overview

JavaScript is single-threaded, so slow work must not block the page. Promises and `async/await` let you handle network requests, timers, and file I/O without freezing the interface.

Key Concepts

- A Promise represents a value that may not be ready yet; `.then/.catch` chain reactions to it.
- `async/await` is syntactic sugar over promises that reads like synchronous code.
- `fetch(url)` returns a promise for a Response; call `.json()` or `.text()` to read the body.
- Always handle errors: failed requests reject, so wrap `await` in `try/catch`.
- The event loop, microtasks, and macrotasks determine when callbacks actually run.
- `AbortController` lets you cancel an in-flight request, useful for search-as-you-type.

Example

```
async function loadCourses() {
  try {
    const res = await fetch('/api/courses');
    if (!res.ok) throw new Error(res.status);
    return await res.json();
  } catch (err) {
    console.error('Failed to load', err);
    return [];
  }
}
```

Summary

Asynchronous JavaScript keeps the UI responsive while the network and other slow operations run. `fetch` with `async/await` and proper error handling is the modern way to talk to APIs.

Practice Checklist

- Fetch a list of items from an API and render them into the DOM.
- Add a loading state and an error state to the same request.
- Use `Promise.all` to load two independent resources concurrently.